

Hiver Customer Support Agent

Technical Report

Brand: AmazonHelp | Dataset: Twitter Customer Support Corpus

Architecture: RAG + 3-Layer Escalation Router

Golden Set: 200 Stratified Examples (60 Cal / 140 Eval)

Repository: github.com/shinesh07/hiver-support-agent

September 2026

1. Problem Framing

For AmazonHelp, "good" means aggressive safety against hallucinated policies and rapid de-escalation of customer frustration. Because e-commerce logistics involve highly sensitive data (tracking numbers, refund statuses, account details), the system must explicitly optimize for Escalation Recall on Ungrounded Cases. If the agent cannot find an exact historical precedent for a policy question, it must escalate to a human rather than risk fabricating an answer.

What I Chose Not to Build

I explicitly did not build a multi-turn state machine or API tool-calling capabilities. Without authenticated access to an actual order management system, an LLM attempting to "resolve" order issues is merely hallucinating plausible-sounding support theater. Instead, this system acts purely as an information-retrieval and triage router — it retrieves historically verified responses and escalates everything it cannot ground.

2. Results vs. Baselines

The system was evaluated against two baselines on a strictly held-out 140-example evaluation set. Statistical significance was verified using McNemar's test on paired predictions.

Metric	Trivial	Simple (k-NN)	Agent	p-value
Intent Accuracy	46.5%	71.8%	88.2%	< 0.001
Ungrounded Recall	14.2%	68.5%	94.7%	< 0.001
Escalation Precision	22.1%	59.3%	81.4%	< 0.01
Escalation Rate	12.0%	45.7%	31.2%	—

Baseline Descriptions

- **Trivial:** Majority-class intent prediction, keyword-based escalation, single canned reply.
- **Simple:** k-NN (k=5) intent classification + Logistic Regression escalation, fitted strictly on the calibration split to prevent data leakage.
- **Agent:** OOD filtering, FAISS dense retrieval with intent filtering, context-budgeted RAG, and a 3-Layer Escalation Router (Hard Regex -> Shrunk Margin Bands -> LLM Judge).

Human vs. LLM-Judge Agreement

To validate the LLM-as-a-judge rubric for Reply Quality, a human evaluator blind-graded 50 responses. Cohen's Kappa (binarized to Acceptable/Poor): Tone k=0.82 (Strong), Helpfulness k=0.74 (Substantial), Faithfulness k=0.65 (Moderate — the LLM judge struggles to penalize subtle omissions compared to a human reviewer).

3. Failure Analysis — Top 5 Failure Modes

FM-1: False Positive OOD on Short Queries

- **Example:** "dm sent"
- **Hypothesis:** The customer is replying to an earlier brand prompt. The text has zero semantic overlap with normal queries, triggering OOD escalation on an empty interaction.

FM-2: Intent Masking via Excessive Politeness

- **Example:** "Hi there! Hope you're having a wonderful day. I was just wondering if someone might check on a tiny issue with my account?"
- **Hypothesis:** Social pleasantries dilute the SentenceTransformer embedding. FAISS retrieves generic "greeting" threads rather than account recovery threads, causing hallucination risk.

FM-3: Sarcasm Bypassing Hard Escalation Rules

- **Example:** "Oh wonderful, another missing package. Give the driver my thanks for throwing it in a puddle."
- **Hypothesis:** VADER registers sarcasm as neutral/positive. Hard regex rules only match explicit anger keywords. The message lands in the "safe" zone instead of the LLM Judge margin band.

FM-4: Shrinkage Over-confidence on Edge Intents

- **Example:** A technical issue regarding the Amazon Prime Video app.
- **Hypothesis:** Very few "technical_support" samples exist. The shrinkage estimator pulled the threshold too aggressively toward the global mean (0.78), letting the agent confidently hallucinate troubleshooting steps.

FM-5: Multi-turn Pronoun Ambiguity

- **Example:** Customer (Turn 3): "Where is it?"
- **Hypothesis:** The pipeline embeds only the latest customer message for retrieval. Without preceding context ("my refund"), FAISS retrieves irrelevant delivery tracking examples.

4. What Is Misleading About My Headline Number?

The headline number (94.7% Safety Recall) is fundamentally misleading due to Information Asymmetry in the Golden Set Labels.

Due to the constraints of the assignment, the 200 Golden Set examples were mock-labeled using heuristic proxies (e.g., if a query contained "refund" and was long, `grounding_available` was set to `False`). The Agent Pipeline's internal logic uses highly correlated features (k-NN distances, regex patterns) to make its escalation decisions. Therefore, the Agent is partially "predicting" the very heuristics that generated the labels, artificially inflating performance metrics compared to what they would be against true, nuanced human annotations.

Furthermore, the McNemar's test assumes independent and identically distributed (i.i.d) samples. Because the evaluation set contains heavily clustered conversational archetypes drawn from a single brand, the true statistical variance is wider than the 95% Confidence Interval implies. The real-world performance would likely be 5-15 percentage points lower than reported.

5. What I Would Do Next With One More Week

- **Human-in-the-Loop Active Learning:** Build a Streamlit labeling UI, dump the 200 mock-labeled examples, and spend 8 hours genuinely hand-annotating the golden set to remove the heuristic bias described above.
- **Contextual Embeddings:** Instead of embedding just the latest message for FAISS, concatenate the last 3 turns with time-decay weighting. This directly solves Failure Mode #5 (Pronoun Ambiguity).
- **Direct Action APIs:** Integrate mock APIs (`get_order_status`, `initiate_refund`) and allow the agent to execute tools rather than relying purely on historical FAISS similarities.
- **Sarcasm Detection Layer:** Add a fine-tuned sentiment classifier (e.g., Twitter-RoBERTa) as a pre-filter before the escalation router to catch sarcastic frustration that VADER misclassifies as neutral.
- **A/B Confidence Calibration:** Implement Platt Scaling on the Agent's raw similarity scores to convert them into calibrated probabilities, enabling principled "I'm X% sure" reasoning.

6. Decision Log (15 Non-Obvious Decisions)

- **1. Brand Selection via Z-Score Normalization:** Selected AmazonHelp by computing a composite score across 4 metrics with Z-score normalization to prevent high-variance metrics from dominating.
- **2. Bypassing $O(N^2)$ Deduplication:** Exact duplicates were non-existent in reconstructed multi-turn threads. Bypassed deduplication in favor of strict CJK/Latin filtering, cutting data prep time by 90%.
- **3. K-Means Stratified Sampling ($k=10$):** Embedded first customer messages and clustered into 10 groups to ensure the golden set covers a diverse range of intents.
- **4. 30% Hard-Case Oversampling:** Built heuristic difficulty detectors and inflated the "hard" ratio to 30% to properly stress-test escalation.
- **5. held_out_ids.json for FAISS Leakage:** Explicitly saved golden set IDs and hard-coded the FAISS builder to exclude them, preventing train-test leakage.
- **6. 3-Layer Cascading Escalation:** Rejected pure LLM-as-router. Built: Hard Regex -> Margin Band -> LLM Judge, reserving expensive calls for ambiguous cases only.
- **7. Shrinkage Estimators for Thresholds:** Shrinks per-intent thresholds toward the global threshold based on N, preventing catastrophic overfitting on rare intents.
- **8. Asymmetric Recall on Ungrounded Cases:** Optimized for Recall strictly on cases lacking historical grounding, where failing to escalate is a catastrophic hallucination risk.
- **9. Graceful Truncation vs. Dropping:** Implemented token-aware truncation of RAG sources instead of dropping lower-ranked results, preserving diversity of evidence.
- **10. McNemar's Test for Significance:** Rejected raw accuracy difference. Implemented paired McNemar's test with 95% CIs to prove outperformance is not random noise.
- **11. Exception Masking from End-Users:** Wrapped LLM calls in safe try-except blocks that log internally but return a polite fallback to the user.
- **12. Strict Calibration Isolation:** The Simple Baseline's LogReg uses a dedicated fit_escalation() method bounded to the 60-example calibration split.
- **13. Robust Markdown Stripping for JSON:** Added regex cleanup before json.loads to handle LLM-generated markdown code blocks around JSON output.
- **14. Single Embedder via Dependency Injection:** Shared one SentenceTransformer instance across all components to prevent memory exhaustion and segfaults.
- **15. tiktoken for Exact Token Counting:** Replaced naive len()/4 with OpenAI's tiktoken to prevent emoji/CJK payloads from silently bypassing the context budget.